Persistent Memory — Team Learning Guide

Logicore (current) vs Claude Code (reference at scale) — extraction + retrieval + storage, with algorithms as discussed

Audience: product + engineering. Goal: one place to understand (1) how our persistent memory works turn-by-turn, (2) where it goes blunt at scale, (3) how Claude Code solves the same problem, (4) what to copy wisely. Sources: **logicore/memory/*** (types, storage, manager, extraction/worker, retrieval/retriever, consolidation/worker) and **D:/claude-code/src/memdir/* + services/extractMemories/* + utils/attachments + query.ts**.

Mental model in one line: Memory = *.md files with YAML frontmatter on disk + a background writer + a scoring reader that injects best files before each turn. No vector DB in either system.

1. Bird-eye loop: Turn-N ends, Turn-N+1 starts

```
Turn-N ends (assistant replied)
  -> WRITE PATH (extraction, background, fire-and-forget)
  -> files land in memory/*.md

Turn-N+1 starts (new user msg)
  -> READ PATH (retrieval, blocking/prefetch, before LLM)
  -> best files injected as <system-reminder>
  -> LLM answers with memory -> loop repeats
```

1.1 Phase-A: after response — submit_for_extraction

- Caller: **await memory.submit_for_extraction(messages + [reply])** — just **queue.put({conversation, session_id, timestamp})**, returns instantly. Reference: **scripts/memory_chat.py:85**.
- Background worker wakes (Logicore throttle 1s; Claude fork fire-and-forget). Builds manifest + transcript, calls LLM, parses, guards, writes to disk + rebuilds/updates MEMORY.md index.
- Current turn gets no benefit. Benefit lands for future turns.

1.2 Phase-B: on new request — inject_context / prefetch

- Caller: **messages = await memory.inject_context(messages, new_msg)** (Logicore, blocking) or Claude **startRelevantMemoryPrefetch()** (non-blocking, runs while model streams).
- Steps: topic detect -> score every header -> budget collect (max 5) -> inject as **<system-reminder>## Relevant Memories...** before last user msg.
- System prompt rule (**prompts.py:860 Check Memory First**): use memory directly, verify path with 1 tool call, never spend 5+ calls rediscovering.
- Consolidation/forgetting runs separately (24h + 5 sessions gate), not per turn.

Owner note: this loop works in MemoryManager, but ChatOrchestrator.run() today only wires in-session OperationalMemory (failure patterns). Persistent loop is only called in scripts/examples. For autonomy it must be BEFORE_TURN -> inject + AFTER_TURN -> submit.

2. Domain concept + what happens after transcript

Domain = label/folder for which aspect: **identity, preferences, relationships, knowledge, domain, additional_learning, operational** (types.py:17). **Kind** = shape: fact/guideline/context/pointer/lesson. **Stability** = rot speed: stable/evolving/volatile/ephemeral. Domain is only for **keep prompt small + rank correctly**.

2.1 Transcript then what? (Logicore extraction/worker.py:370)

```
transcript = last 20 msgs x 500 chars joined
text = transcript lowercased
score[domain] = count(DOMAIN_KEYWORDS[domain] in text)
relevant = domains with score >= top//2
manifest = only headers in relevant (max 40), else all
prompt = SYSTEM + MANIFEST + TRANSCRIPT -> LLM -> JSON ops
```

Example S01-Turn-1 ('Never edit legacy/, keep parser API compatible' + FAIL AssertionError): **never** hits preferences, **test/bug** hits knowledge, **FAIL** hits operational. So relevant=[knowledge, preferences, operational]. First run store empty: manifest='No existing memories'. Second run: manifest shows 1-2 lines like **- never-touch-legacy.md [preferences/guideline]: Never edit legacy/... [tags]**. LLM returns create/update/delete ops. That manifest+transcript pair is the real input — transcript alone is useless.

2.2 Same idea on retrieval (retriever.py:87,267,363)

```
topic = keyword match query -> primary/secondary/intent
ranking = decay*0.6 + importance*0.25 + confidence*0.15
if domain==primary: *=1.5; elif secondary: *=1.2
budget: 3 primary + 1 secondary + 1 overflow (max 5), 4KB/file, 60KB/session
```

So **parser-fix [knowledge]** beats **user-name [identity]** even if newer — domain boost dominates. Without domain, newest always wins.

3. Pitfalls causing blunt memory + LLM cost

3.1 Extraction (worker.py)

- Transcript 20x500 truncates tool outputs, loses cause-effect.
- Manifest filter substring keyword (failing != failure, fix missing) — paraphrase hides right files -> duplicates.
- LLM per turn + second merge call; queue sleep 1s lags; scripts even block on queue.join().
- Silent drops: malformed JSON / bad domain dropped via Pydantic, no feedback.
- Grounding regex crude: rejects short valid facts, accepts long hallucinations.
- Dedup Jaccard 0.5 on LLM tags — inconsistent tags => duplicates + wrong merges.
- Filename from LLM: collision/overwrite/traversal risk.

3.2 Retrieval (retriever.py)

- Keywords collected but never scored — only domain_match boosts. Content relevance ignored.
- Decay-forget dead code: floor=0.3+imp*0.4 >=0.3 always, but drop only if decay<0.2 — never fires. Same bug in consolidation forget_stale. Only maxAge deletes.
- Magic numbers 0.6/0.25/0.15, boost 1.5/1.2, budget 3/1/1 — untuned; overflow starved.
- surfaced_this_session never re-surfaces; 60KB cap; 4KB truncation mid-sentence.
- LLM topic + select = +2 calls on critical path if enabled; off = blunt keyword path.

3.3 Storage / system

- Flat glob scan O(n) each call, mtime cache races, 200 cap drops oldest silently, 25KB MEMORY.md stale.
- Global ~/.logicore/memory — no per-project isolation. S08 vendor key vk_live_... would persist as plain md. No redaction.

- Consolidation gate 24h+5 sessions may never run; rule path deletes exact-description dup only; LLM path 200x4000 chars = overflow + cost.
- Not wired in ChatOrchestrator; no save/recall/forget tools; OperationalMemory (in-session) confused with persistent; prompt says 'NOT a file — don't use read_file' but model still tries.

Net: keyword instead of semantic + LLM glue everywhere + dead forgetting + rigid budgets = more files, more noise, more cost, less precision.

4. Claude Code storage pattern (verified at scale)

```
~/.claude/projects/<sanitized-git-root>/memory/
  MEMORY.md  # index only, 200 lines / 25KB
  user_role.md / feedback_*.md / project_*.md
  team/MEMORY.md + *.md
  logs/YYYY/MM/YYYY-MM-DD.md  # KAIROS append-only
```

- Per-project isolation by canonical git root; worktrees share. Priority: COWORK_OVERRIDE > settings autoMemoryDirectory (trusted) > default. Enabled by default; off via DISABLE_AUTO_MEMORY / --bare / CCR-no-storage / settings.
- File = md + minimal frontmatter name,description,type[user|feedback|project|reference]. parseMemoryType undefined-tolerant. @include depth 5, HTML stripped, ~40KB soft max.
- MEMORY.md 1-line hooks <150ch, no frontmatter, never body. truncateEntrypointContent cuts at line then last newline before 25KB + warning.
- Scan: readdir recursive, read first 30 lines, parse, newest-first, cap 200, single-pass stat+read, allSettled. mtime threaded for 'saved X ago' without restat.
- Proposal files (memoryMetadata/domain/kind, topicDetection, clusterRetrieval, decayScoring, contradictionDetection) exist but prod recall does NOT use them — only tests import them. Prod = Sonnet sideQuery selector.
- Safety: validateMemoryPath + realpathDeepestExisting symlink check; 30+ gitleaks secret scan + redact for team; team sync per-repo hash, ETag 304, delta by hashes, 200KB batch, 412 retry x2 local-wins, fs.watch debounced 2s.
- Layers: CLAUDE.md instruction vs memdir durable vs transcript — never one store. WHAT_NOT_TO_SAVE blocks derivable code/git/fix-recipes/ephemeral even on explicit ask.

5. Extraction in practice — Claude fork vs Logicore JSON

5.1 Claude: LLM does, not describes

- Fork = same system prompt + prompt-cache share, cheap. maxTurns 5. Taught: turn1 all Reads parallel, turn2 all Writes parallel.
- Jail createAutoMemCanUseTool: Read/Grep/Glob free, Bash read-only, Edit/Write only inside memoryDir.
- Cursor lastMemoryMessageUuid + modelVisible(user|assistant) count; mutual exclusion hasMemoryWritesSince (skip fork if main saved); throttle turns counter; inProgress -> pendingContext coalesce to 1 trailing run; drain 60s on shutdown; skipTranscript true.

Minimal example. Main: U1 'don't mock DB — burned last quarter' / A1 'used real DB, pass'. Manifest empty. Fork prompt: 'Analyze last ~2 msgs + Types + What NOT to save + How to save (Step1 file, Step2 MEMORY.md line)'. Fork Turn1: Write feedback_db_mock.md (rule+Why+How). Turn2: Edit MEMORY.md add '- [DB testing](feedback_db_mock.md) — use real DB'. Parent moves cursor, toasts 'Memory saved'. If main had already Written prefs.md same turn, fork skipped entirely.

5.2 Logicore: LLM describes, Python writes

```
queue.put({conversation}) -> transcript(20x500) + domain-filtered manifest(40)
-> chat([system,user]) -> JSON [{action,filename,metadata,body}]
-> regex extract + Pydantic validate + _is_grounded + Jaccard>0.5 -> store.write + update_index()
```

Aspect	Claude fork	Logicore JSON
Who writes	LLM via tools after Read	LLM describes, Python writes
Context	full fork + cache + UUID cursor	truncated string
Manifest	full list	keyword-domain filtered (can hide)
Dedup	read body then decide	Jaccard tags
Overlap	mutual exclusion + 1 trailing	queue + sleep + 3x requeue
Cost	2-4 cache-hit tool turns	1 big chat + merge + retries

6. MEMORY.md new vs old building

- New: ensureMemoryDirExists mkdir -p ahead; system prompt 'MEMORY.md currently empty'; fork manifest empty, no Reads; Step1 Write topic file, Step2 Edit MEMORY.md add hook.
- Old: system prompt injects truncated MEMORY.md; fork manifest lists all; fork MUST Read candidates parallel then Write/Edit parallel; topic-organized not chronological; Thursday -> 2026-03-05 absolute; remove outdated; keep hooks <150ch; beyond 200 lines not loaded.
- Logicore difference: update_index() auto-rebuilds grouped by domain in Python; Claude index edited by agent via Edit. Hence Logicore dedup without reading body.

7. Domain filtering in extraction?

No — not in shipped Claude. extractMemories.ts:398 manifest = formatMemoryManifest(all, cap 200). memoryScan header has no domain/kind/stability. topicDetection/clusterRetrieval/decayScoring imported only by their own tests (grep verified), never by extract/find/query/attachments. They are proposal docs. Filtered axis is **scope (auto vs team/)**, not domain. Logicore _build_manifest + _infer_domains filtering is Logicore-only — and the blunt source when keywords miss.

8. Algorithm: Claude extraction (as-shipped pseudocode)

```
on query-loop end (final text, no tools) -> executeExtractMemories fire-and-forget
state: cursor, inProgress, turnsSince, pendingContext, inFlight
runExtraction(ctx):
  if !enabled or remote or subagent: return
  if inProgress: pendingContext=ctx; log coalesced; return
  newCount = count(user|assistant after cursor)  # fallback all if cursor lost
  if main Wrote memory since cursor: cursor=last uuid; log skipped_direct_write; return
  if !trailing:
    turnsSince++
    if turnsSince < (tengu_bramble_lintel ?? 1): return
  turnsSince=0; inProgress=true
  headers = scanMemoryFiles(dir)  # 30 lines, newest-first, 200 cap
  manifest = "- [type] file (iso): desc"
  prompt = opener(newCount,manifest)+TYPES+WHAT_NOT_TO_SAVE+HowToSave(2-step)
  result = runForkedAgent(prompt, cacheShare, jailedTools, maxTurns:5, skipTranscript:true)
  # fork: T1 Reads parallel, T2 Writes/Edits parallel + MEMORY.md hook
  if success: cursor=last uuid  # else keep for retry
  written = extractWrittenPaths(result)
  log tokens/cache_hit/files/duration; toast if non-index written
  inProgress=false
  if pendingContext: runExtraction(pending, trailing=true)  # only msgs after new cursor
```

Failure best-effort log only. No JSON parsing, no domain filter, no decay. Dedup by Reading.

9. Trigger: after how many msgs?

Every turn by default, not every N messages. **stopHooks.ts:157** fires each loop end. Gate is turn counter: **turnsSinceLastExtraction < (tengu_bramble_lintel ?? 1)** (GrowthBook, default 1). =1 every turn, =3 every 3rd turn (prompt then says Analyze last ~6 msgs). **newMessageCount** only scopes prompt + cursor, not gate. Trailing runs skip throttle. Example default: Turn1 2 new -> run cursor=msg2; Turn2 2 new -> run cursor=msg4.

10. Algorithm: Claude retriever (prefetch, never blocks)

```
A. system prompt (cached once): MEMORY.md index + Types + WhenToAccess + BeforeRecommending(verify file/grep
B. per-turn prefetch at turn start, parallel with streaming:
  if !enabled or single-word or sessionTotal>=60KB: return
  sessionTotal = scan msgs for relevant_memories attachments  # compact resets
  headers = scan (30 lines, 200 cap); candidates = headers - alreadySurfaced
  selected = sideQuery Sonnet 256tok JSON {selected_memories[]}:
    sys: certain-only (unsure->skip, none->[]; tool docs in use->skip, warnings->keep)
    usr: Query + manifest + recentTools
    filter validFilenames; fail->[]
  bodies = read 4KB/file truncate+note + freshness header
  dedup vs readFileState + alreadySurfaced, slice 5
  consume post-tools if settled else skip-next-iter; using-dispose aborts
```

Example. Memories db.md / role.md / freeze.md. Query 'why did checkout tests fail on CI?' Manifest+query -> ['db.md'] certain, others skipped. Read 4KB, inject <system-reminder>Memory (saved 2d ago): ...use real DB. Why: mock/prod divergence...>. Next turn won't re-pick db.md. Caps: 5/turn, 4KB/file, 60KB/session.

11. Storage mechanism + how diffs are made + diff vs Logicore

- Write path: harness mkdir; main or fork Write/Edit (Edit needs prior Read); 2-step file + MEMORY.md hook; topic-organized; absolute dates; remove outdated. No Python rebuilder.
- Read path: scan 30-line headers, manifest, truncation 200/25KB, mtime for freshness text (not ranking).
- Index: agent-maintained 1-line hooks; beyond cap not loaded with warning.
- Variants: team/ own MEMORY.md + sync; KAIROS daily logs append-only, nightly /dream distills to index.

Aspect	Claude	Logicore
Root	per-project slug	global
Frontmatter	name,desc,type	+ domain/kind/conf/imp/stab/tags
Index	agent Edits hooks	Python rebuild by domain
Writer	forked tools maxTurns 5	JSON + Python write
Dedup	Read body	Jaccard tags
Secrets	gitleaks block+redact	none
Session cap	scan msgs, compact resets	counter

12. Direction to update Logicore wisely

- Isolate per-project path (canonical git root), keep global only as fallback; add settings + env priority + hardened validator.
- Replace JSON-ops with tool-jailed sub-agent (Read/Grep/Glob free, Write/Edit only memoryDir, Bash read-only, maxTurns 5, parallel Reads then Writes) sharing prompt cache; keep Pydantic only as fallback.

- Replace 20x500 transcript string with UUID cursor + modelVisible count; add mutual exclusion (skip if main Wrote) + coalesce to 1 trailing + throttle turns flag (default 1) + drain on shutdown.
- Send full manifest (200 cap) to writer; drop keyword-domain prefilter; enforce WHAT_NOT_TO_SAVE + Rule/Why/How + absolute dates; agent Edits MEMORY.md 1-line hooks, Python only truncates on read.
- Retrieval: prefetch non-blocking parallel with streaming; Sonnet-certain-only selector 256tok + recentTools anti-noise; 5/4KB/60KB caps with compact-aware accounting (scan msgs); freshness text not score; fix decay floor bug (floor makes decay<0.2 impossible) — use age-only forget + ranking only.
- Add secret scan block/redact, /memory edit + /dream consolidate entry points, verify-before-recommend (file exists/grep) + ignore-means-empty semantics, per-agent @-mention isolation, telemetry selection-rate denominator.
- Expose save/recall/forget tools + wire BEFORE_TURN inject + AFTER_TURN submit in ChatOrchestrator; keep domain/decay as ranking helper, not gate.

End of guide. Update order suggested: (1) per-project paths + validator + caps, (2) cursor + mutual exclusion + coalescing, (3) fork writer + jail + prompts, (4) prefetch selector + accounting fix + decay fix, (5) secrets + tools + wiring + evals.